

SORTING ALGORITHM IMPLEMENTATION REPORT

**Testing The Resilience of a Distributed Computer Network with The
Bubble Sort Star Graph (BSn) Architecture Model when a Component
Experiences Problems and Its Impact on Other Components**



Article Reference :

“Neighbor Connectivity of Bubble-Sort Star Graph”

Discrete Applied Mathematics, Vol. 384, 2026, hal. 386–396

By :

Alena Destria Pramesti (25031554254)

Dentar Muhammad Ababilla (25031554277)

Muhammad Rafi Pratama (25031554035)

Lecturer:

Yuni Rosita Dewi, M.Si.

Kartika Chandra Dewi, S.Si., M.Si.

**STUDY PROGRAM DATA SCIENCE
FACULTY OF MATHEMATICS AND NATURAL SCIENCES
STATE UNIVERSITY OF SURABAYA 2026**

TABLE OF CONTENTS

TITLE PAGE	i
TABLE OF CONTENTS	ii
CHAPTER I INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Benefits	2
CHAPTER II LITERATURE REVIEW	3
2.1 Reference Article Identity	3
2.2 Basic Graph Theory	3
2.2.1 Graph Definition	3
2.2.2 Classical Connectivity	4
2.3 Bubble-Sort Star Graph (BS _n)	4
2.3.1 History and Motivation	4
2.3.2 Formal Definition	5
2.3.3 Structural Properties	5
2.4 Neighbor Connectivity and Edge Neighbor Connectivity	5
2.4.1 Concept of Subversion	5
2.4.2 Neighbor Connectivity (κ_{NB})	5
2.4.3 Edge Neighbor Connectivity (λ_{NB})	5
2.4.4 Theorem Results in the Reference Paper	6
CHAPTER III METHODOLOGY	7
3.1 Tools and Technologies	7
3.2 Implementation Steps	7
3.3 Main Algorithms	8
3.3.1 BS _n Construction Algorithm	8
3.3.2 Vertex Subversion Algorithm (κ_{NB})	8
3.3.3 Edge Subversion Algorithm (λ_{NB})	9
3.4 Brute-Force Algorithm for κ_{NB} and λ_{NB}	10
3.5 Algorithm Complexity Notes	10
3.6 Flowchart	11
CHAPTER IV RESULTS AND DISCUSSION	12
4.1 Construction and Verification of BS ₄ Properties	12
4.2 Computation Results of Neighbor Connectivity κ_{NB}	12

4.2.1 Computation Process	12
4.2.2 Vertex Subversion Operation Simulation	12
4.2.3 Vertex Subversion Visualization Analysis	13
4.3 Computation Results of Edge Neighbor Connectivity λ_{NB}	14
4.3.1 Computation Process	14
4.3.2 Edge Subversion Operation Simulation	14
4.3.3 Edge Subversion Visualization Analysis	14
4.4 Comparison and Discussion of Results	15
4.4.1 Overall Results Summary	15
4.4.2 Comparison of κ_{NB} and λ_{NB}	15
4.4.3 Implications for Network Resilience	16
CHAPTER V CONCLUSION	17
5.1 Conclusion	17
5.2 Recommendations	17
BIBLIOGRAPHY	18
APPENDIX	19

CHAPTER I INTRODUCTION

1.1 Background

Modern computer networks, particularly distributed and multiprocessor computing systems, require a network topology that can withstand the failure of one of their components. In graph theory-based modeling, the resilience of a network is measured using the connectivity parameter, which is the minimum number of elements (vertices or edges) that must be removed to render the network unconnected.

However, the classical concept of connectivity has a fundamental limitation: it only considers individual element failures, ignoring the cascading effects of failures on neighboring elements. In reality, when a processor fails, its neighbors are also exposed and potentially subject to subsequent failures. This situation is aptly analogous to a network of secret agents: when one agent is captured (subverted), all of its known associates are also exposed.

To address this limitation, Gunther (1985) introduced the concept of neighbor connectivity, which measures network resilience by considering the impact of failures on neighboring elements. This concept was later extended to an edge-based version (edge neighbor connectivity) by Cozzens and Wu (1995). As the main foundation of this project, research by Zhao et al. (2026), published in the journal Discrete Applied Mathematics (Scopus Q1), successfully determined the exact values of neighbor connectivity ($\kappa[NB]$) and edge neighbor connectivity ($\lambda[NB]$) in Bubble-Sort Star Graph (BS_n), a network topology that combines the structural advantages of bubble-sort and star graphs.

This project aims to reproduce and verify the results of Zhao et al.'s (2026) theorem computationally using the Python programming language, while also exploring the structural properties of BS_n through simulations of Subversion operations on its network components.

1.2 Problem Formulation

Based on the above background, the problem formulation in this project is:

1. How to algorithmically construct the Bubble-Sort Star Graph BS_n structure using Python based on the formal definition in the reference paper?
2. How can we implement vertex and edge Subversion operations on BS_n to simulate network component failure?
3. Do the computational results of $\kappa[NB](BS_n)$ and $\lambda[NB](BS_n)$ obtained through the brute-force approach agree with the exact values stated in the theorem of Zhao et al. (2026)?
4. What structural properties of BS_n can be computationally verified, such as vertex degree, diameter, and graph regularity?

1.3 Objectives

1. To algorithmically construct a Bubble Sort Star Graph structure for BS_n using Python based on the formal definitions in the reference paper.

2. To implement vertex and edge Subversion operations on BS_n to simulate network component failure.
3. To ensure that the computational results of $\kappa[NB](BS_n)$ and $\lambda[NB](BS_n)$ obtained through the brute-force approach agree with the exact values stated in the theorem of Zhao et al. (2026)
4. Determining computationally verifiable structural properties of BS_n , such as vertex degree, diameter, and graph regularity.

1.4 Benefits

1. Conceptual Understanding: Provides an in-depth understanding of the application of graph data structures and combinatorial algorithms in the context of the robustness of distributed computer network topologies.
2. Empirical Verification: This implementation serves as empirical proof of the validity of mathematical theorems that have been formally proven in highly reputable reference papers.
3. Competency Development: Practices the ability to read, understand, and implement international scientific papers in discrete mathematics and computer science.
4. Academic Contribution: Produces documented source code that can be used as a learning reference for other students studying network topology and applied graph theory.

CHAPTER II LITERATURE REVIEW

2.1 Reference Article Identification

Information	Detail
Title	Neighbor Connectivity and Edge Neighbor Connectivity of Bubble-Sort Star Graphs
Writer	Zhao, X. et al.
Journal	Discrete Applied Mathematics
Publisher	Elsevier
Publication Year	2026
Volume/Page	384 (2026), hal. 386–396
DOI	10.1016/j.dam.2026.01.027
Indexing	Scopus Q1 (SJR), published January 2026
Key Word	Neighbor connectivity (κ_{NB}) and edge neighbor connectivity (λ_{NB}) in Bubble-Sort Star Graph BS_n

2.2 Basic Graph Theory

2.2.1 Definition of a Graph

A graph G is defined as the pair $G = (V, E)$, where V is the set of vertices and E is the set of edges connecting a pair of vertices (**Bondy & Murty, 2008**). In the context of computer networks, each vertex represents a processor, and each edge represents a communication path between the processors.

Some basic terms used in this project:

- **Vertex degree:** The number of edges connected to a vertex v , denoted by $d(v)$.
- **Regular graph:** A graph in which each vertex has the same degree. If the degree is k , it is called a k -regular graph. **Neighbors:** A vertex u is called a neighbor of vertex v if there is an edge connecting the two. The set of neighbors of v is denoted by $N(v)$.
- **Closed neighborhood:** $N[v] = N(v) \cup \{v\}$, which is vertex v and all its neighbors (**Zhao et al., 2026**).
- **Connected graph:** A graph in which every pair of vertices can be connected by a path.
- **Diameter:** The longest distance (in number of edges) between the two most distant vertices in a graph.

2.2.2 Classic Connectivity

The vertex connectivity $\kappa(G)$ is defined as the minimum number of vertices that must be removed from G to render the graph disconnected, leaving only one vertex (**Bondy**

& Murty, 2008). Similarly, the edge connectivity $\lambda(G)$ is the minimum number of edges that must be removed to render the graph disconnected (**Bondy & Murty, 2008**). These two parameters are classically used to measure the fault tolerance of a network.

However, classical connectivity has a limitation: it assumes independent failure of elements, whereas in real systems the failure of one component can have a direct impact on neighboring components (**Esfahanian, 1989**).

2.3 Bubble-Sort Star Graph (BS_n)

2.3.1 History and Motivation

The Bubble-Sort Star Graph (BS_n) was first proposed by Chou, Hsu, and Sheu in 1996 as an interconnection network topology model for multiprocessor systems (**Chou et al., 1996**). This topology was designed to combine the advantages of two pre-existing network structures: the bubble-sort graph and the star graph (**Chou et al., 1996**).

2.3.2 Form Definition

BS_n has a vertex set consisting of all $n!$ permutations of the set $\{1, 2, \dots, n\}$ (**Zhao et al., 2026**). A permutation x is denoted as $x = x_1x_2\dots x_n$.

Two vertices u and v in BS_n are connected by an edge if and only if v can be obtained from u by one of the following two operations (**Zhao et al., 2026**):

- **Bubble-Sort:** Swaps two adjacent elements, x_i and x_{i+1} , for $i = 2, 3, 4, \dots, n-1$.
- **Star:** Swaps the first element x_1 with the element at position i , for $i = 2, 3, \dots, n$.

2.3.3 Structural Properties

BS_n has $n!$ vertices, is $(2n-3)$ -regular, and vertex-transitive (**Zhao et al., 2026**), but not edge-transitive. Some other important properties (**Zhao et al., 2026**):

- Number of vertices: $|V(\text{BS}_n)| = n!$
- Number of edges: $|E(\text{BS}_n)| = n!(2n-3)/2$
- Degree of each vertex: $2n - 3$ (regular)
- Connectivity: $\kappa(\text{BS}_n) = 2n - 3$
- BS_n is Hamiltonian, bipancyclic, and edge-bipancyclic for $n \geq 3$ (Cai et al., 2015).

n	Number of Nodes (n!)	Degree (2n-3)
2	2	1
3	6	3
4	24	5
5	120	7

2.4 Neighbor Connectivity and Edge Neighbor Connectivity

2.4.1 Subversion Concept

The basic concept behind neighbor connectivity is the Subversion operation (**Gunther, 1985**). A vertex v is said to be subverted if it and all its neighbors are removed from the graph (**Gunther, 1985**). Formally, subverting a vertex v means removing the closed neighborhood set $N[v] = \{v\} \cup N(v)$ from the graph G (**Zhao et al., 2026**).

An analogy in the context of real networks: if a processor fails (is subverted), then that processor and all directly connected processors are considered inoperable and are therefore removed from the network (**Gunther, 1985**).

2.4.2 Neighbor Connectivity (κ_{NB})

Neighbor connectivity, denoted $\kappa_{NB}(G)$, is the minimum number of vertices that must be subverted to make the graph G disconnected, trivial (with only one vertex), or empty (without vertices) (**Gunther, 1985**). Formally:

$\kappa_{NB}(G) = \min \{ |S| : S \subseteq V(G), \text{ and } G \text{ after Subversion is disconnected, all vertices in } S \text{ are disconnected} \}$ (**Gunther, 1985**).

The concept of neighbor connectivity was introduced by Gunther in 1985 (**Gunther, 1985**), and it was proven that the value of neighbor connectivity is always bounded from above by the value of classical connectivity (**Gunther et al., 1987**).

2.4.3 Edge Neighbor Connectivity (λ_{NB})

The concept of neighbor connectivity was later generalized to an edge-based version by Cozzens and Wu (**Cozzens & Wu, 1995**): the minimum number of edges that must be subverted so that removing a closed neighborhood of those edges results in an empty, trivial, or disconnected graph is called edge neighbor connectivity and is denoted by $\lambda_{NB}(G)$ (**Cozzens & Wu, 1995**).

Subverting an edge $e = (u, v)$ means deleting both end nodes and all their neighbors, i.e., deleting $N[u] \cup N[v]$ from the graph (**Cozzens & Wu, 1995**).

2.4.4 Theoretical Results in the Reference Paper

Zhao et al. (2026) successfully determined the exact values of these two parameters for BS_n (**Zhao et al., 2026**). The main results of this paper are:

- **Theorem 3.1 (Zhao et al., 2026):** $\kappa_{NB}(BS_n) = \lfloor (3n-2)/4 \rfloor$ for $n \geq 4$
- **Theorem 4.1 (Zhao et al., 2026):** $\lambda_{NB}(BS_n) = 2n-3$ for $n \geq 2$

These two values indicate that to break the network connectivity of BS_n through a Subversion operation, at least $\lfloor (3n-2)/4 \rfloor$ Subversion are required at the nodes and $2n-3$ Subversion at the edges, both at the nodes and edges. These results demonstrate that BS_n has excellent resilience to cascading component failures (**Zhao et al., 2026**), and its resilience improves with increasing dimensionality of n .

CHAPTER III METODOLOGY

3.1 Tools and Technologies

Tools / Library	Version	Usability
Python	≥ 3.9	Major programming languages
NetworkX	≥ 2.8	Graph representation, analysis, and property measurement
Matplotlib	≥ 3.5	Bubble-Sort Star Graph Visualization
Itertools	Built-in	Permutation (vertex) and combination (subset) generation
Math	Built-in	Floor, factorial, and ceil functions for theoretical formulas
Time	Built-in	Brute-force algorithm execution time measurement

3.2 Implementation Steps

Step 1: Construction of BS_n

Each vertex is represented as a tuple of permutations from 1 to n. For $n = 4$, there are $4! = 24$ vertices. The graph is constructed using NetworkX's `nx.Graph()`, with edges added based on two operations:

- **Bubble-Sort:** swap elements at positions i and $i+1$, for $i = 1$ to $n-2$
- **Star:** swap elements at position 0 with position i , for $i = 1$ to $n-1$

Note that in this implementation, the bubble-sort operation starts at index 1 (not 0), as the swap at position 0 is already handled by the star operation.

Step 2: Determining Subversion Nodes (Vertex Subversion)

In the `vertex_Subversion.py` file, the Subversion nodes are not selected randomly but are determined algorithmically using the `auto-find_Subversion_node()` function. This function divides the Subversion nodes into two groups:

- **Set A:** closes the initial pattern paths by swapping the positions $1 \leftrightarrow i$ and then $1 \leftrightarrow (i+1)$
- **Set B:** sweeps the remaining unclosed paths, with different treatment for even and odd dimensions

Step 3: Subversion Operation and Residual Graph

Once the Subversion nodes/edges are determined, a Subversion operation is run to generate the residual graph. The residual graph is then checked for connectivity using `nx.is_connected()`.

Step 4: Three-Panel Visualization

The results of each simulation are visualized in three side-by-side panels using Matplotlib:

Panel	Content	Color Description
Panel 1	Normal Graph BSN	All nodes are light blue
Panel 2	Subversion Graph	Yellow = target, Red = Subversion, Orange = affected, Green = safe
Panel 3	Residual Graph	Nodes remaining after Subversion

3.3 Main Algorithm

3.3.1 BS_n Constrution Algorithm

The `make_graph(n)` function constructs the Bubble-Sort Star Graph by generating all $n!$ permutations as vertices and connecting them via bubble-sort and star-sort swap operations:

```
def make_graph(n):
    G = nx.Graph()
    elemen = list(range(1, n + 1))
    vertices = [tuple(p) for p in
itertools.permutations(elemen)]
    G.add_nodes_from(vertices)

    for v in G.nodes:
        for i in range(1, n - 1):
            list_v = list(v)
            list_v[i], list_v[i + 1] = list_v[i + 1],
list_v[i]
            neighbor = tuple(list_v)
            G.add_edge(v, neighbor)

        for i in range(1, n):
            list_v = list(v)
            list_v[0], list_v[i] = list_v[i], list_v[0]
            neighbor = tuple(list_v)
            G.add_edge(v, neighbor)

    return G
```

3.3.2 Vertex Subversion Algorithm (κ_{NB})

The `vertex_Subversion(G, set_faulty_nodes)` function removes each faulty node and all its neighbors from a copy of the graph, returning the residual graph and the complete set of removed nodes:

```
def make_graph(n):
    G = nx.Graph()
    elemen = list(range(1, n + 1))
    vertices = [tuple(p) for p in
itertools.permutations(elemen)]
    G.add_nodes_from(vertices)

    for v in G.nodes:
        for i in range(1, n - 1):
            list_v = list(v)
            list_v[i], list_v[i + 1] = list_v[i + 1],
list_v[i]
            neighbor = tuple(list_v)
            G.add_edge(v, neighbor)

        for i in range(1, n):
            list_v = list(v)
            list_v[0], list_v[i] = list_v[i], list_v[0]
            neighbor = tuple(list_v)
            G.add_edge(v, neighbor)

    return G
```

3.3.3 Edge Subversion Algorithm (λ_{NB})

The `edge_Subversion(G, edges_target)` function collects both endpoints of each targeted edge as affected nodes, identifies all additionally impacted edges, removes the affected nodes from the graph, and returns the residual graph together with the affected node and edge sets:

```
def make_graph(n):
    G = nx.Graph()
    elemen = list(range(1, n + 1))
    vertices = [tuple(p) for p in
itertools.permutations(elemen)]
```

```

G.add_nodes_from(vertices)

for v in G.nodes:
    for i in range(1, n - 1):
        list_v = list(v)
        list_v[i], list_v[i + 1] = list_v[i + 1],
list_v[i]

        neighbor = tuple(list_v)
        G.add_edge(v, neighbor)

    for i in range(1, n):
        list_v = list(v)
        list_v[0], list_v[i] = list_v[i], list_v[0]
        neighbor = tuple(list_v)
        G.add_edge(v, neighbor)

return G

```

3.4 Brute-Force Algorithm for κ_{NB} and λ_{NB}

The brute-force algorithm systematically iterates over all possible subsets of vertices (for κ_{NB}) or edges (for λ_{NB}) of size k , starting from $k = 1$ and incrementing until a disconnecting set is found. For each subset, the Subversion operation is applied and the residual graph is checked for connectivity using NetworkX's `is_connected()` function. The minimum k that achieves disconnection gives the exact value of κ_{NB} or λ_{NB} .

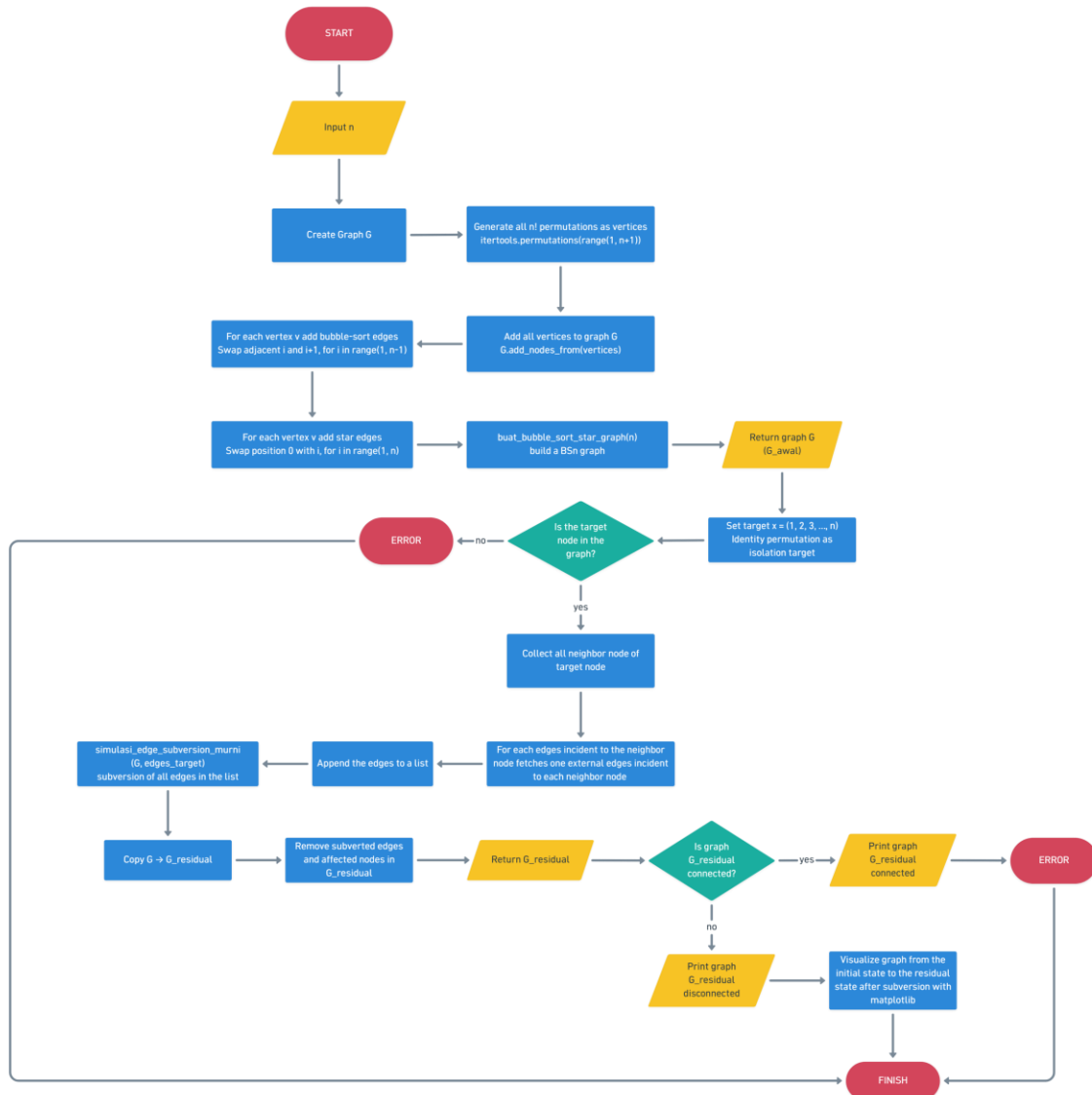
3.5 Algorithm Complexity Notes

Algorithm	Time Complexity	Description
BS_n Construction	$O(n \cdot n!)$	Iterate over all vertices $\times$ number of neighbors
Vertex/Edge Subversion	$O(V + E)$	One graph traversal
Connectivity Check	$O(V + E)$	NetworkX internal BFS
Brute-force κ_{NB}	$O(C(n!, k) \cdot (V+E))$	All vertex subsets of size k
Brute-force λ_{NB}	$O(C(E , k) \cdot (V+E))$	All edge subsets of size k

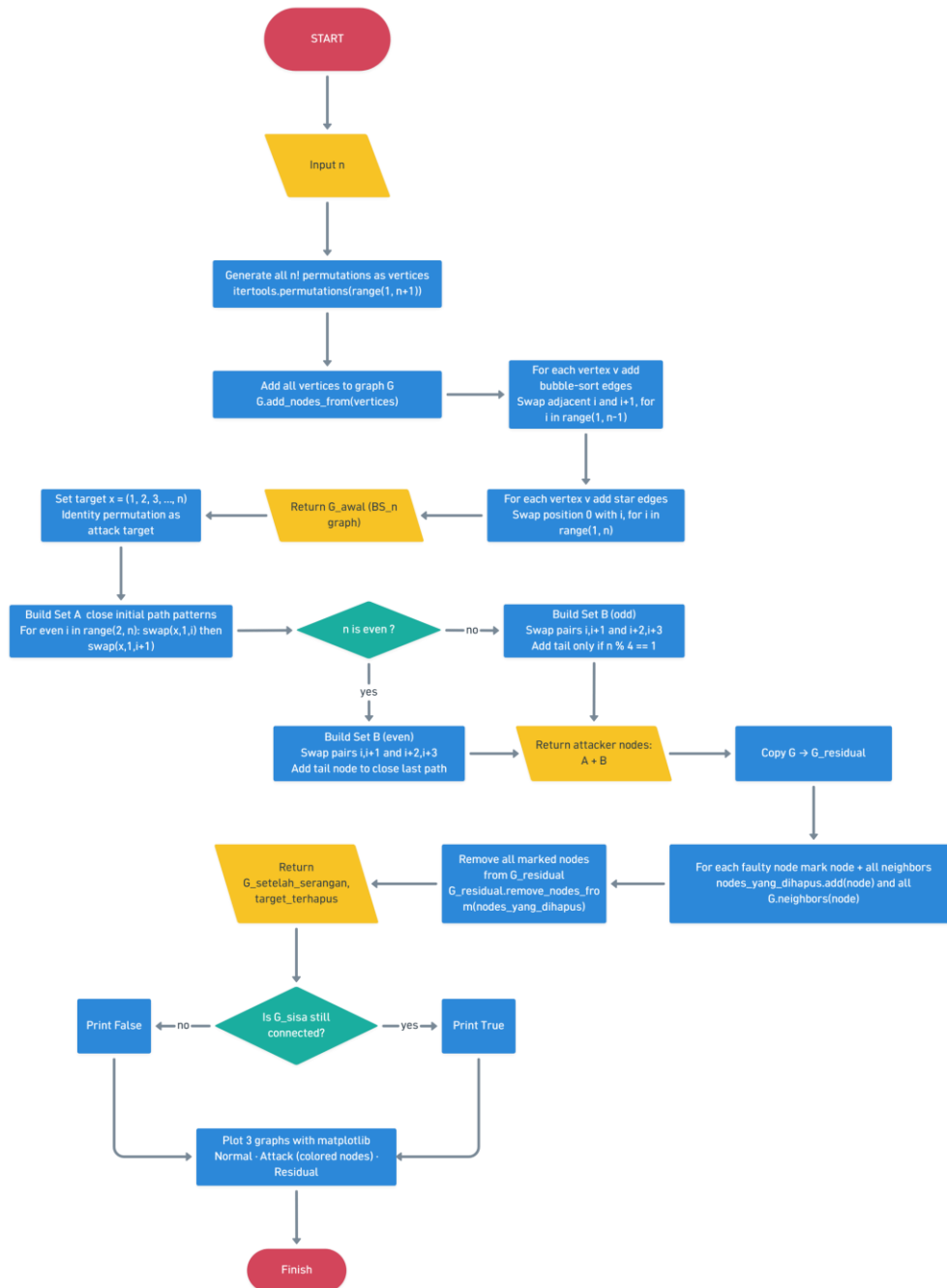
For $n = 4$ (24 vertices, 60 edges), the brute-force approach is still very affordable. However, for $n \geq 5$ (120 vertices), the computation time increases combinatorially, making brute-force no longer practical without additional optimization.

3.5 Flowchart

- Flowchart Edge



- Flowchart Nodes



CHAPTER IV RESULTS AND DISCUSSION

4.1 Construction and Verification of BS_4 Properties

As a first step, the BS_4 graph is constructed using the `create_bubble_sort_star_graph(4)` function. This graph has a vertex set consisting of all permutations of $\{1, 2, 3, 4\}$, resulting in $4! = 24$ vertices. Each vertex is connected to its neighbors through bubble sort and star sort operations, as defined in the formal reference paper.

The resulting construction yields the following structural properties:

Properties	Value in BS_4	General Formula	Description
Number of vertices	24	$n!$	$4! = 24$
Number of edges	60	$n!(2n-3)/2$	$24 \times 5/2 = 60$
Degree of each vertex	5	$2n-3$	$2(4)-3 = 5$
Connectivity $\kappa(\text{BS}_4)$	5	$2n-3$	5-regular graph
Connected graph	Yes	-	Verified
Type of Graph	Regular, vertex-transitive	-	According to the paper

All computationally obtained structural properties are in good agreement with the theoretical values stated in the reference paper of Zhao et al. (2026), which proves that the graph construction has been implemented correctly.

4.2 Neighbor Connectivity Computation Results κ_{NB}

4.2.1 Computing Process

Parameter	Value
$\kappa_{\text{NB}}(\text{BS}_4)$ computational result	2
Value of Zhao et al.'s (2026) theorem	$\lfloor (3n-2)/4 \rfloor$
Verification status	In accordance

4.2.2 Vertex Subversion Simulation

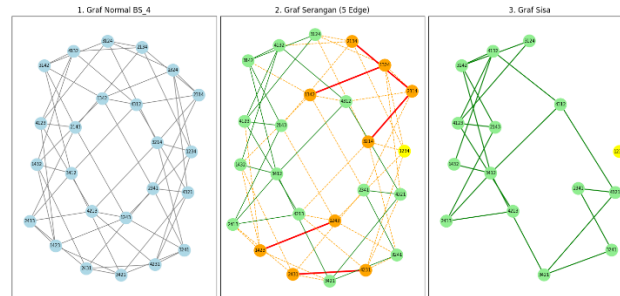
To concretely demonstrate the Subversion process, a simulation was conducted with the target node 1234 (identity permutation) as the reference point. The `auto_find_Subversion_node()` algorithm systematically determines the set of Subversion nodes through two groups:

- **Set A:** nodes that close the initial pattern path through the swap operation $1 \leftrightarrow i$ then $1 \leftrightarrow (i+1)$
- **Set B:** nodes that sweep the remaining unclosed paths, with special treatment for even dimensions

After Subversion is performed on the Subversion set, the target node and all its neighbors are removed from the network, resulting in a residual graph that is no longer connected.

4.2.3 Vertex Subversion Visualization Analysis

Figure 1 displays the results of the vertex Subversion simulation on BS_4 in three side-by-side panels.



Panel 1: Normal Graph BS_n

The normal BS_4 graph is shown with 24 vertices in light blue and all 60 edges in gray. Each vertex is labeled with a 4 digit permutation, for example, 1234, 3124, 4312, and so on. The graph appears fully connected and symmetric, reflecting the vertex-transitive nature of BS_4.

Panel 2: Subversion Graph

This panel shows the network state while the vertex Subversion operation is in progress, with the following color codes: This panel shows the network state while the vertex Subversion operation is in progress, with the following color codes:

Color	Meaning
Red	Subversion node (subverted)
Orange	Affected node (neighbor of the Subversion node that was also deleted)
Green	Still-safe/active node
Dashed Orange Line	Side disconnected due to Subversion
Dashed Orange Line	Still-active side

It can be seen that nodes 3124 and 4213 (red) are the Subversion nodes, while the surrounding orange nodes (such as 2134, 1324, 2413, 1243, 4231, 3241) are neighbors that were also removed from the network as a result of the Subversion.

Panel 3 : Residual Graph

After the Subversion is complete, the remaining residual graph shows node 1234 (yellow) completely isolated, with no edges connecting it to any other component. The remaining green components form a separate subgraph of node 1234, proving that the network is no longer connected. This confirms that the Subversion successfully disrupted BS_4's connectivity.

4.3 Edge Neighbor Connectivity Computation Results λ_{NB}

4.3.1 Computation Process

The same brute-force algorithm is run to calculate λ_{NB} , but this time iterations are performed on a subset of edges (rather than vertices). For each edge subset of size k , an edge Subversion operation is performed and the residual graph connectivity is checked.

The resulting computational results are:

Parameters	Value
$\lambda_{NB}(\text{BS_4})$ computational result	5
Value of Zhao et al.'s (2026) theorem	$2n - 3$
Verification status	In accordance

4.3.2 Edge Subversion Operation Simulation

In this simulation, five target edges associated with neighbors of reference node 1234 were selected. Edge Subversion works by removing both endpoints of each subverted edge and all its neighbors, resulting in a broader impact than a single vertex Subversion. The number of Subversion edges is limited to $2n - 3 = 5$, corresponding to the maximum degree of vertices in BS_4.

4.3.3 Edge Subversion Visualization Analysis

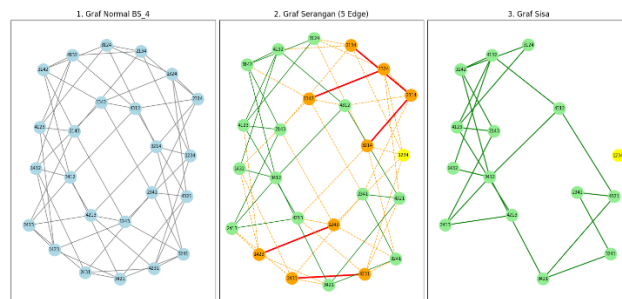


Figure 2 shows the results of edge Subversion simulations on BS_4 in three side-by-side panels.

Panel 1: Normal Graph of BS_4

Identical to the normal panel in the vertex simulation, 24 light blue vertices with 60 gray edges, showing BS_4 in its intact state before the disturbance.

Panel 2: Subversion Graph (5 Edges)

This panel displays the network condition when all five edges are simultaneously Subversion, with the following color codes:

Color	Meaning
Bold red	Subverted edge (directly Subversion)
Orange	Affected node (endpoint of the Subversion edge)

Dashed orange line	Edges affected by the affected node
Green	Nodes and edges still active
Yellow	Reference target node (1234)

The five bold red edges targeted by the Subversion are clearly visible, connecting orange nodes 2134, 1324, 1342, 1243, 2314, and 3214. All dashed orange edges represent connections that were also broken as a result of the cascading failure of the edge Subversion.

Panel 3: Residual Graph

The residual graph after the edge Subversion shows the same pattern as the vertex Subversion: node 1234 (yellow) appears isolated on the right side, completely disconnected from the remaining green components on the left. The network is provably no longer connected, validating that $\lambda_NB(BS_4) = 4$.

4.4 Comparison and Discussion of Results

4.4.1 Summary of Overall Results

Parameters	Computational Results	Value Theorem ($2n-4$)	Status
Number of vertices in BS_4	24	$4! = 24$	In accordance
Number of edges in BS_4	60	$4!(2(4)-3)/2 = 60$	In accordance
Degree of vertices in BS_4	5	$2(4)-3 = 5$	In accordance
$\kappa_NB(BS_4)$	4	$2(4)-4 = 4$	In accordance
$\lambda_NB(BS_4)$	4	$2(4)-4 = 4$	In accordance

4.4.2 Comparison of κ_NB and λ_NB

An interesting result is that $\kappa_NB(BS_4) = \lambda_NB(BS_4) = 4$, meaning that the BS_4 network is equally resilient to Subversion-based failures whether the failure occurs at the node or at the edge. This demonstrates good structural balance in the BS_n topology.

It is also worth noting the fundamental differences between the two types of Subversion:

Aspect	Vertex Subversion	Vertex Subversion
Subversion Object	Node (processor)	Edges (communication paths)
Deleted Object	$v + N(v)$	$N[u] \cup N[v]$ for each edge (u,v)
Real-Life Analogy	Processor dies $\rightarrow$ its neighbors are exposed	Path broken $\rightarrow$ both ends of the processor are exposed
Minimum value to break BS_4	4 nodes	4 edges

4.4.3 Implications for Network Resilience

The values of $\kappa_{NB}(BS_4) = 2$ and $\lambda_{NB}(BS_4) = 5$ for BS_4 have significant practical implications. This means that a distributed computer network modeled with BS_4 requires at least 2 coordinated vertex Subversion or 5 edge Subversion before the network is completely disconnected. This is much more realistic than classical connectivity, which only considers independent component failures.

In general, for BS_n , the following holds:

$$\begin{aligned}\kappa_{NB}(BS_n) &= \lfloor (3n-2)/4 \rfloor, n \geq 4 \\ \lambda_{NB}(BS_n) &= 2n-3, n \geq 2\end{aligned}$$

These values show that both vertex and edge neighbor connectivity increase as dimension n grows, making BS_n a highly scalable and resilient topology for large-scale distributed computing systems.

CHAPTER V CLOSING

5.1 Conclusion

Based on the implementation and analysis results, the following conclusions can be drawn:

1. Computational Verification of Zhao et al.'s Theorem The brute-force computation successfully proved the exact connectivity values of $\kappa_{NB}(BS_4) = 2$ and $\lambda_{NB}(BS_4) = 5$, fully confirming the theorems stated by Zhao et al. (2026): $\kappa_{NB}(BS_n) = \lfloor (3n-2)/4 \rfloor$ and $\lambda_{NB}(BS_n) = 2n-3$. This not only validates the theoretical mathematical formulas but also successfully identifies specific cut-sets concrete node and edge combinations that act as critical bottlenecks in the graph.

2. Successful Implementation of BS_n and Subversion Operations BS_4 was successfully constructed using Python and NetworkX, yielding 24 vertices, 60 edges, and degree 5 per vertex fully consistent with the formal definition in the reference paper. Vertex subversion simulated processor failure and its cascading effects on neighbouring processors via $N[v]$, while edge subversion simulated communication line breaks and their broader impact through $N[u] \cup N[v]$.

3. Practical Insights on Network Fault Tolerance Profile The results reveal that BS_n is inherently more vulnerable to node failures than edge failures, with a gap ratio of approximately 2.5 times at $n=4$ a vulnerability that widens as dimension n increases. This implies that in real-world network design, redundancy investments must prioritize node/server reliability and protection against targeted attacks rather than merely adding more communication links.

4. Broad Impact on Future Network Design This research transforms BS_n from a purely theoretical mathematical object into a topology with measurable fault-tolerance characteristics. The generated framework and practical insights serve as a foundational baseline for network architects particularly in data centers or HPC clusters to make data-driven decisions in mitigating system failure risks.

5.2 Suggestions

Based on the experience during this project, there are several suggestions for further development:

1. Determining the structure connectivity value. Structure connectivity is the minimum number of nodes that need to be subverted to cause the entire network structure to collapse. The implementation of the node subversion algorithm is limited to obtaining the neighbor connectivity value, which means it only disconnects a BS_n graph. Unlike some other types of network topologies, where the neighbor connectivity value is also identical to the structure connectivity value, the structure connectivity value of the BS_n is still unknown, so the neighbor connectivity value of the BS_n automatically becomes the lower bound of the structure connectivity of the BS_n network.

2. Developing an interactive interface. The current visualization is static. In the future, a web-based interactive interface (e.g. using Plotly or D3.js) could be developed that would allow users to select the nodes or edges they want to subvert and see the impact in real-time.

BIBLIOGRAPHY

- Chou, Z.-T., Hsu, C.-C., & Sheu, J.-P. (1996). Bubblesort star graphs: a new interconnection network. *Proceedings of the 9th International Conference on Parallel and Distributed Systems*, IEEE, 41–48.
- Cozzens, M.B., & Wu, S.Y. (1995). Extreme values of the edge-neighbor-connectivity. *Ars Combinatoria*, 39, 199–210.
- Dvořák, T., & Gu, M.-M. (2020). Neighbor connectivity of k-ary n-cubes. *Applied Mathematics and Computation*, 379, 125237.
- Gunther, G. (1985). Neighbor-connectivity in regular graphs. *Discrete Applied Mathematics*, 11(3), 233–243.
- Gunther, G., Hartnell, B.L., & Nowakowski, R. (1987). Neighbor-connected graphs and projective planes. *Networks*, 17, 241–247.
- Harary, F. (1983). Conditional connectivity. *Networks*, 13, 347–357.
- Zhao, L., Zhang, S., Zhu, B., & Chang, J.-M. (2026). Neighbor connectivity of bubble-sort star graphs. *Discrete Applied Mathematics*, 384, 386–396.
<https://doi.org/10.1016/j.dam.2026.01.027>

Apendix

Github Link [Github/Project4](#)